

```
# HumAIne Swarm ,Ã Conversational MLOps Assistant Reference
## RAG Knowledge Document for HumAIne Swarm Chatbot
```

```
**Project:** HumAIne Swarm (Conversational AI for Human-Machine
Collaborative MLOps)
**Published:** DCOSS-IoT 2025 (DOI: 10.1109/DCOSS-IoT65416.2025.00162)
**Docker Hub:** `gfatouros/humaine-swarm:latest`
**HumAIne project:** Grant Agreement 101120218
```

1 ,Ã What is HumAIne Swarm

The HumAIne Swarm Assistant is an intelligent conversational agent that enables researchers and developers to interact with the HumAIne project's MLOps infrastructure through natural language. It supports:

- **Documentation Q&A** ,Ã answers questions about the HumAIne project, its architecture, deliverables, and technical components using RAG over indexed project documents
- **Kubeflow management** ,Ã list, inspect, run, and monitor ML pipelines through conversation
- **MinIO storage interaction** ,Ã browse and retrieve pipeline artifacts, models, datasets, metrics
- **ML workflow support** ,Ã guidance on setting up pipelines, troubleshooting, and comparing run results

The Swarm Assistant operationalises the Swarm Learning paradigm: LLM agents coordinate and delegate to specialised tools, collectively providing more capable assistance than any single agent could.

2 ,Ã Architecture

2.1 Technology Stack

Component	Technology
LLM	OpenAI GPT-4.1-mini
Agent Framework	LangChain + LlamaIndex
UI	Chainlit (chat interface, port 8000)
Vector DB (RAG)	Pinecone
MLOps	Kubeflow
Object Storage	MinIO
Authentication	Keycloak (OAuth)
Language	Python (>=3.11, <3.13)
Package Management	Poetry

2.2 Architecture Layers

...

User (Chainlit Chat)
,Üi


```
### 3.4 ML Workflow Support
- Guide users through setting up and running ML pipelines
- Provide information on available components, models, datasets
- Troubleshoot common pipeline issues
```

4 ,Ã Setup and Deployment

4.1 Local Development

```
```bash
1. Clone and configure
cp env.sh.example env.sh
Edit env.sh: add OpenAI key, Pinecone key, MinIO credentials, Kubeflow
host

2. Source environment
source env.sh

3. Install (requires Python >=3.11, <3.13 and Poetry)
poetry install

4. Run
chainlit run app.py -w
Available at: http://localhost:8000
```
```

4.2 Docker (Pre-built Image)

```
```bash
Pull image
docker pull gfatouros/humaine-swarm:latest

Configure
cp .env-example .env
Edit .env:
OPENAI_API_KEY=...
PINECONE_API_KEY=...
OAUTH_KEYCLOAK_CLIENT_SECRET=...
KUBEFLOW_HOST=http://huanew-kubeflow.ddns.net/pipeline

Run
docker run -d \
 --name humane-swarm-assistant \
 -p 8000:8000 \
 --env-file .env \
 --restart unless-stopped \
 gfatouros/humaine-swarm:latest
```
```

4.3 Key Environment Variables

| Variable | Purpose |
|----------|---------|
|----------|---------|

```
|---|---|
| `OPENAI_API_KEY` | GPT-4.1-mini inference |
| `PINECONE_API_KEY` | Vector database for RAG |
| `OAUTH_KEYCLOAK_CLIENT_SECRET` | Authentication |
| `MINIO_ENDPOINT` | MinIO storage (default: s3-minio.humaine-horizon.eu)
|
| `KUBEFLOW_HOST` | Kubeflow pipeline host |
| `LOG_LEVEL` | Logging verbosity (default: ERROR) |
```

5 ,Ã RAG Indexing Guidelines

5.1 What to Index for Hackathon Support

For the KPI-2.3 hackathon experiment, the following documents should be indexed into Pinecone in this priority order:

| Priority | Document | Reason |
|----------|---|---|
| 1 | `humaine_rag_technical_reference.md` | Hackathon-specific: exact API patterns with HumAIne rationale |
| 2 | `humaine_al_hackathon.ipynb` | Exact variable names and notebook structure |
| 3 | `humaine_explainerdashboard_api_reference.md` | Complete XAI API reference |
| 4 | `humal_api_reference.md` | HumAL AL architecture and workflow |
| 5 | `D5.4_Training_Centre_Development.pdf` | Project context for AL and XAI |
| 6 | `D5.1_Platform_Integration.docx` | HumAIne platform architecture |
| 7 | `text_explanation_20newsgroups_demo.ipynb` | Reference implementation |
| 8 | `library_components_overview.md` | XAI library overview |

5.2 Chunking Recommendations

For optimal RAG retrieval on these documents:

- **Chunk size:** 500,Ã800 tokens
- **Overlap:** 100,Ã150 tokens
- **Metadata to store per chunk:** `source\_file`, `section\_title`, `document\_type` (reference/notebook/deliverable)
- **Do NOT chunk code blocks mid-function** ,Ã keep complete function definitions together

5.3 Test Queries After Indexing

Validate the RAG setup by querying the chatbot with these before the hackathon:

...

1. "Show me the modAL ActiveLearner constructor for LogisticRegression with entropy_sampling"
2. "Which query strategy for 4 classes with 12 initial labels and why?"

3. "How do I wrap a trained learner in ClassifierExplainer? Why use learner.estimated not learner?"
4. "Write a predict_fn closure for LIME on TF-IDF text ,Â what are the common mistakes?"
5. "How do I display explainer.plot_importances() in Colab?"
6. "What is the relationship between HumAL's dispatch labeling and modAL's query-teach cycle?"
7. "What SHAP backend does ClassifierExplainer select for LogisticRegression?"
8. "How does entropy_sampling differ from uncertainty_sampling for multi-class problems?"

Each answer should reference HumAL's architecture and produce code with the exact variable names from the hackathon notebook (`X\_pool`, `y\_pool`, `X\_test\_df`, `vectorizer`, `feature\_names`, `TEAM\_NAMES`).

6 ,Â Example Conversations (Expected Quality)

Query: "Which classifier should I use?"

Expected answer: Use `LogisticRegression(max\_iter=1000)`. TF-IDF vectors are high-dimensional and sparse ,Â LogisticRegression handles this setting best. It natively supports `predict\_proba()` needed for uncertainty query strategies, and selecting it means `ClassifierExplainer` will use `shap.LinearExplainer` ,Â fast and exact for linear models. This aligns with HumAL's design for the ticketing pilot.

Query: "Show me the ActiveLearner constructor"

Expected answer:

```
```python
from modAL.models import ActiveLearner
from modAL.uncertainty import entropy_sampling
from sklearn.linear_model import LogisticRegression
```

```
learner = ActiveLearner(
 estimator=LogisticRegression(max_iter=1000),
 query_strategy=entropy_sampling,
 X_training=X_initial,
 y_training=y_initial
)
```
```

Query: "Write predict_fn for LIME"

Expected answer:

```
```python
def predict_fn(texts):
 X_sparse = vectorizer.transform(pd.Series(texts).fillna(''))
 X_df = pd.DataFrame(X_sparse.toarray(), columns=feature_names)
 return learner.predict_proba(X_df)
```
```

With explanation: ``fillna('')`` handles None/empty strings, ``toarray()`` converts sparse to dense, ``predict_proba`` returns probabilities needed by LIME.

7 ,Ã Citation

```bibtex

@INPROCEEDINGS{11096208,

author={Fatouros, George and Makridis, Georgios and Kousiouris, George and

Soldatos, John and Tsadimas, Anargyros and Kyriazis, Dimosthenis},

booktitle={2025 21st International Conference on Distributed Computing in

Smart Systems and the Internet of Things (DCOSS-IoT)},

title={Towards Conversational AI for Human-Machine Collaborative MLOps},

year={2025},

pages={1079-1086},

doi={10.1109/DCOSS-IoT65416.2025.00162}

}

```

HumAIne EU Project ,Ã Grant Agreement 101120218

For RAG indexing into HumAIne Swarm chatbot